

HoneyCon Network

B.Sc. (Hons) Computer Science School of Computing and Communications
Lancaster University



By Thomas Green
24.04.2026

I certify that the material contained in this dissertation is my own work and does not contain unreferenced or unacknowledged material. I also warrant that the above statement applies to the implementation of the project and all associated documentation.

Regarding the electronically submitted work, I consent to this being stored electronically and copied for assessment purposes, including the School's use of plagiarism detection systems in order to check the integrity of assessed work.

I agree to my dissertation being placed in the public domain, with my name explicitly included as the author of the work.

Name: Thomas Green

Date: 2026-04-24

Abstract

The aim of this project is to develop a scalable, dynamic honeypot infrastructure through the use of Docker containers and OpenFlow. Honeypots are vital tools for cybersecurity allowing for the simulation of real systems to attract and analyze attacks. Traditional honeypot systems are limited by their static nature (fixed configurations - making them predictable) and they also lack scalability (resource constraints - deploying multiple traditional honeypots can require a significant amount of resources). HoneyCon seeks to overcome these limitations by creating a system that can dynamically scale honeypot instances in response to real-time malicious activity. OpenFlow will allow traffic to be routed to honeypots. By the end of the project, HoneyCon will provide a scalable solution for capturing a range of attacks efficiently while reducing resource waste.

Contents

1 Introduction.....	6
1.1 Honeypots: An Overview.....	6
1.2 Aims of the System.....	6
1.3 Objectives.....	7
1.4 Structure.....	8
2 Background.....	9
2.1 Traditional Security Measures.....	9
2.2 Role of Honeypots in Cybersecurity.....	9
2.3 Related Work.....	10
2.3.1 T-POT The All In One Multi Honeypot Platform.....	10
2.3.2 Improvements provided by HoneyCon.....	10
3 Methodology.....	11
3.1 System Design.....	11
3.1.1 Requirements Specification.....	11
3.1.2 Network Topology.....	11
3.2 Proposed Architecture.....	13
3.2.1 Detection Layer.....	13
3.2.2 Orchestration Layer.....	14
3.2.3 Deception Layer.....	14
3.3 Implementation Challenges.....	15
3.3.1 Proper Network Redirection.....	15
3.3.2 Framework Deprecation.....	15
3.3.3 Justification for Hybrid Approach.....	16
3.4 Resource Required.....	17
3.4.1 Hardware.....	17
3.4.2 Software.....	17
4 Development.....	18
4.1 System.....	18
4.1.1 Tools and Packages.....	18
4.2 Honeypot Deployment with Docker.....	19
4.3 Traffic Management with OpenFlow.....	20
4.4 Automation and Scaling.....	21
5 Testing and Evaluation.....	22
5.1 Functional Testing.....	22
5.2 Performance Evaluation.....	23
6 Conclusion.....	25
6.1 Review of Project Aims.....	25
6.2 Future Work.....	25

6.3 Lessons Learned.....	25
6.4 Changes from the Proposal.....	26
6.5 Final Remarks.....	26
Appendix A.....	27
Appendix B.....	28
References.....	33

1 Introduction

1.1 Honeypots: An Overview

Honeypots are network decoys that are designed to attract attackers and prevent them from getting to important information, they allow the observation of attack behaviors in a controlled environment. As networks become larger and attacks more complex, traditional honeypots face scalability issues, it then becomes difficult to handle higher traffic volumes or distributed attacks that can cause serious issues.

This project aims to address these limitations by combining containerization and SDN technologies. Docker [9] containers will allow dynamic scaling of honeypot instances. OpenFlow will be used for traffic routing, making sure that malicious traffic is directed to the correct honeypot appropriate to the attack. The system will be capable of automatically adjusting to traffic loads, deploying new honeypots without manual intervention.

By using open-source tools, this project aims to create a solution for scaling honeypot networks, improving the ability to capture and analyze a range of cyberattacks without making it too abstract.

1.2 Aims of the System

With the relentless increase in cyber attacks in the modern day, the demand for systems that can combat and analyze malicious activities has become critical. Traditional honeypots struggle to keep up with the pace of the ever-evolving attack methods that malicious entities come up with due to their static nature, this leaves important systems and networks vulnerable to large scale attacks that can inflict major damage. Traditional honeypots face issues with scalability; higher volumes of traffic and distributed attacks pose a serious threat when the security measures can't keep up with the attacks. HoneyCon focuses on creating a lightweight yet scalable honeypot infrastructure through the use of Docker containers and OpenFlow. By using containerization and SDN technologies, the system allows rapid deployment, efficient resource utilization with flexible scaling depending on the scale of the attack. This is very important for managing excessive use of resources; computing resources are limited, if a system is overwhelmed by malicious attacks, the system cannot fulfill its role in combating the threat. A system that can't dynamically adjust to the varying level of attack is also at risk of underutilization during periods of low activity. HoneyCon will build upon the existing traditional honeypot framework by integrating containerization. Unlike static honeypot deployments that

require manual set up and constant maintenance, HoneyCon dynamically scales honeypot instances depending on traffic conditions on the network.

1.3 Objectives

The overarching aim for HoneyCon is to develop a dynamic honeypot infrastructure and explore the monitoring capabilities.

- **Develop and deploy a honeypot network:** Using Docker to manage lightweight honeypots to be scaled upon varying network traffic conditions. HoneyCon will use Docker containers to create lightweight honeypots. Docker offers an environment where each honeypot is isolated, a minimal amount of resources is consumed allowing for optimal scalability. Each honeypot will serve a different purpose to provide coverage against a range of attacks.
- **Integrate OpenFlow for managing traffic:** OpenFlow is to be integrated into HoneyCon for the ability to manage how traffic is routed, redirecting malicious traffic to the appropriate honeypots for further analysis by manipulating flow tables on the network switches using OpenFlow controllers.
- **Automate scaling of honeypots:** an automated scaling system is incorporated into HoneyCon, by monitoring the volume of traffic, HoneyCon can be scaled appropriately. In high traffic volume situations, the system will automatically deploy additional honeypots to meet the demands of the surge in volume. When the traffic volume decreases, the system will be scaled down, honeypots are removed to free up resources. This is done without manual intervention.
- **Evaluate the system's effectiveness:** One of the key functions is to capture and log data of malicious activity. Each honeypot container has the capabilities of recording all interactions. These logs provide valuable information into how attackers think and what vulnerabilities they are trying to target. It will also be valuable to see how efficient in terms of resources that this system is dealing with varying network traffic volumes.

1.4 Structure

This paper is structured as follows:

Chapter 2 - Background

Provides an overview of existing technologies, their limitations and how honeypots complement them. Also discusses related work on scalable honeypot deployments.

Chapter 3 - Methodology

This chapter details the structure of HoneyCon and the techniques used. It also clarifies the main design choices and the reasoning behind them.

Chapter 4 - Development

This covers the development of the system, this includes the system and software being used, the setting up of honeypots, network configurations, and mechanisms for traffic redirection.

Chapter 5 - Testing and Evaluation

Presents the results and observations of the systems performance in relation to its scalability, resource utilization and effectiveness in detecting malicious traffic.

Chapter 6 - Conclusion

Analyzes whether the system was successful in meeting the objectives, including suggestions for possible improvements and what knowledge has been gained from building the system.

2 Background

In order to frame the scope of HoneyCon, examining existing work in this field provides an understanding on how the existing systems function, their limitations, and how HoneyCon can provide improvements.

2.1 Traditional Security Measures

Cybersecurity has traditionally relied on tools such as Firewalls, Intrusion Detection Systems (IDS) and much more for the safeguarding of networks from malicious actors. A firewall acts as a filter, separating an internet network from an external network. Incoming and outgoing traffic is filtered based on predefined rules. [1]. The problem with static rule based detection is that it creates an opening for zero-day attacks, making the firewall completely useless. On the other hand, Intrusion detection systems monitor the network activity for any signs of suspicious behaviour, the issues with IDS come down to the high false positive and false negative rate. IDS frequently struggle to distinguish between legitimate and malicious traffic, this can cause unnecessary downtime or employee negligence. In companies with an employee size of 500 to 1499, 27% of all notifications are ignored or never investigated [2]. This is most likely due to fatigue and negligence, checking on alerts that are false most of the time can lead to this. This obviously creates a massive hole in the security wall for attackers to penetrate.

2.2 Role of Honeypots in Cybersecurity

In the cybersecurity sector, honeypots offer a more interactive approach and act as decoy systems to lure in potential attackers through the simulation of vulnerabilities. Honeypots allow for the strengthening of defences for integral systems by providing invaluable data collected from attack attempts. The types of honeypots vary, there are 3 main types: Low-Interaction, High-Interaction, Pure. Low-Interaction honeypots are designed specifically to be lightweight and only simulate basic services; this type is often used as an early warning system. High-Interaction honeypots are designed to replicate a systems services as close as possible, the attackers are provided with more intractability, this also provides the system with more data about the attacker allowing for better prevention of attacks, these honeypots do consume more resources and are time consuming to maintain. Pure honeypots are completely passive and don't generate any traffic on its own, it is a full-scale system that waits for an attacker to make the first move then logs all the actions. This type of honeypot is not easy for attackers to detect which makes them suitable for gathering and analyzing attack behaviours. [3]

2.3 Related Work

This section will cover related works with real world examples.

Traditional Honeypots: Systems such as Honeyd are used to simulate vulnerabilities in a network, this is done by trapping attackers and logging all their activities on the network. These systems are difficult to scale; they become resource hungry when the attack volume increases [4]. Existing systems often face difficulties with scalability, traditional honeypots are designed for static deployment, this means that they struggle to scale up or down depending on the volume of traffic.

2.3.1 T-POT The All In One Multi Honeypot Platform

T-POT is an open source system developed by Deutsche Telekom, it supports 20+ honeypots with many visualization options[5]. T-POT is a well established framework, it uses docker to deploy several honeypots simultaneously, while it runs a predefined amount of honeypots, it does not scale dynamically.

2.3.2 Improvements provided by HoneyCon

Compared to already existing systems like T-POT, HoneyCon offers several improvements.

- **Automated Scaling:** HoneyCon automates the process of adding or removing honeypots by using OpenFlow controllers in response to the network conditions. This improvement ensures that resources are efficiently handled, allowing for the system to adapt to varying traffic volumes.
- **Flexibility:** While T-POT uses containers to deploy honeypots, HoneyCon further improves on this by combining container orchestration with network traffic management [6]. This provides the flexibility to quickly create or destroy containers based on the network conditions.
- **Lightweight:** HoneyCon is built to be lightweight, by using Docker containers, there is no need for complex server setups that typically come with running a traditional honeypot.

In conclusion to this section, HoneyCon builds upon existing honeypots, the key factor is that it introduces several improvements for the scale that it is built for. The goal of HoneyCon is to be dynamic yet lightweight.

3 Methodology

This section outlines the approach taken to design, develop, and evaluate the HoneyCon system. The methodology is structured to ensure a systematic implementation of a scalable honeypot infrastructure using Docker containers and OpenFlow-based SDN. The process is divided into distinct phases: system design, implementation, automation, testing, and evaluation.

3.1 System Design

The design of HoneyCon is based on integrating Software Designed Networking (SDN) and containerization to create a dynamic and responsive approach to network security. Traditional honeypots are static systems that often have static configurations which makes it very easy for malicious actors to identify that it is a trap, allowing them to completely avoid them. HoneyCon is designed to exist in a dormant state until the network conditions indicate that there is an active threat. Once an active threat has been detected, HoneyCon is then activated by scaling up or down honeypot containers depending on the incoming traffic.

3.1.1 Requirements Specification

To make sure that HoneyCon is effective and resource efficient the following requirements have been set.

Functional Requirements: The HoneyCon system must autonomously detect malicious activity without manual intervention and upon detection dynamically deploy dockerized cowrie instances.

Non-Functional Requirements: To sustain the deception that the honeypot is a legitimate network, the redirection of traffic must be transparent to the malicious actor. HoneyCon must also minimize the latency between the breach of the threshold and the deployment of decoy instances.

3.1.2 Network Topology

The network topology used for HoneyCon is a star topology, this is a very simple topology with the goal of focusing on the core functionalities of honeypot management and traffic redirection while minimizing routing complexity. By utilizing a star topology, it simplifies the application of flow rules; the controller is only required to manage a single switch to redirect traffic. This approach would allow near instant updates to the flow table when the SYN packet threshold is reached, making it more responsive to the bursts of traffic.

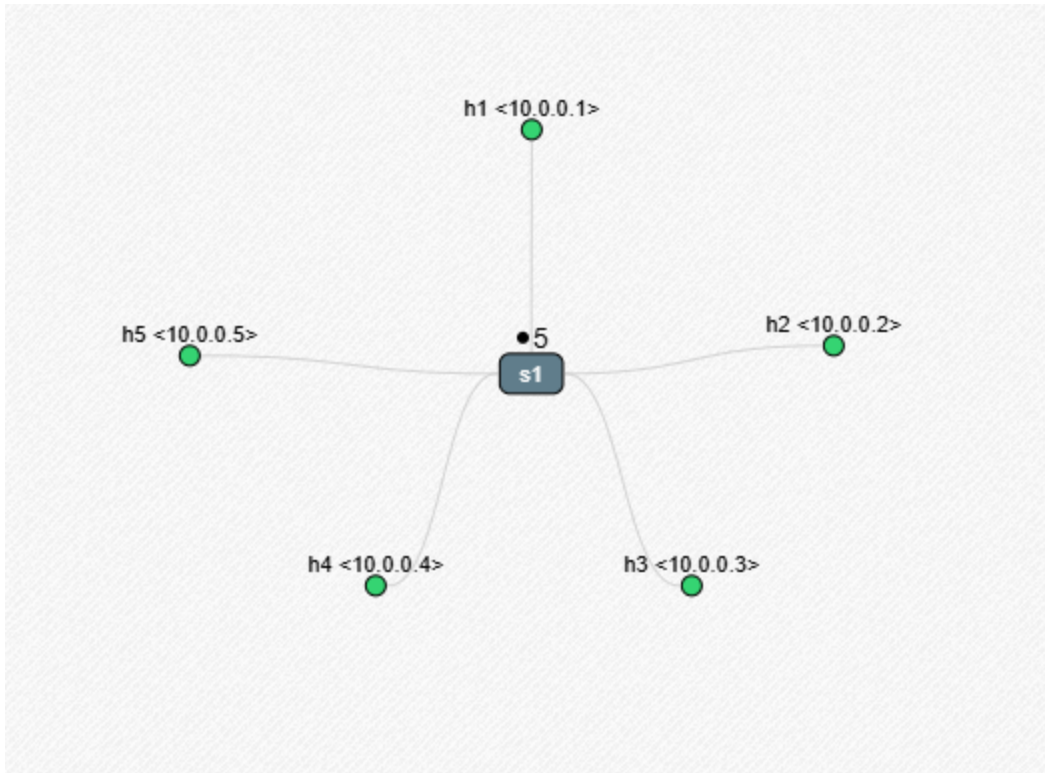


Figure 3.1.2: Mininet [10] topology visualization. [8]

3.2 Proposed Architecture

HoneyCon is a modular design, featuring a 3 tier framework: high-level network management, orchestration, and low-level containerization. This is achieved by utilizing Software-Defined Networking (SDN). HoneyCon separates the packet forwarding layer from the control logic, allowing for real-time monitoring and deployment of cowrie honeypot containers.

3.2.1 Detection Layer

The first stage of the HoneyCon architecture is the Detection Layer, which is responsible for monitoring all incoming traffic and identifying potential threats to the network. This layer is composed of the Data Plane - implemented using Open vSwitch, and the traffic ingestion logic that receives and processes the incoming packets from the Data Plane to determine whether the connection is legitimate or malicious. As shown in Figure 3.2.1, the system captures incoming packets, then determines whether they are malicious, if they are found to be malicious they are redirected to a honeypot container without being made aware, where they can be safely observed.

[illegible]

Figure 3.2.1: controller.py terminal output - showing traffic detection and redirection.

Traffic Monitoring

In Figure 3.2.1, the controller.py script implements a reactive model dependent on the traffic that is received. Upon receiving a connection that meets the criteria, the Open vSwitch creates a PacketIn message. The terminal output demonstrates successful interception of packets and the redirection of those packets to one of the cowrie containers that have been created.

3.2.2 Orchestration Layer

The second stage of the architecture is the Orchestration Layer. This layer is responsible for automating the scaling of the deployment of honeypots based on real-time telemetry received from the network.

```
(osken-env) tomgx@ubuntu:~/typ$ python honeypot-scaling.py
SYN Count: 22 | Active Containers: 1
DEPLOYED: ef0963abf96f on port 2223. Total: 2
SYN Count: 15 | Active Containers: 2
SYN Count: 13 | Active Containers: 2
SYN Count: 14 | Active Containers: 2
SYN Count: 19 | Active Containers: 2
SYN Count: 7 | Active Containers: 2
SYN Count: 20 | Active Containers: 2
DEPLOYED: 60e2eb825696 on port 2224. Total: 3
SYN Count: 10 | Active Containers: 3
SYN Count: 8 | Active Containers: 3
SYN Count: 3 | Active Containers: 3
SYN Count: 8 | Active Containers: 3
SYN Count: 0 | Active Containers: 3
REMOVED: Honeypot on port 2224. Total: 2
SYN Count: 0 | Active Containers: 2
REMOVED: Honeypot on port 2223. Total: 1
SYN Count: 0 | Active Containers: 1
REMOVED: Honeypot on port 2222. Total: 0
```

Figure 3.2.2: honeypot-scaling.py - shows the automated scaling of honeypot containers.

Dynamic Scaling

As shown in Figure 3.2.2, the scaling script maintains a constant watch over the network state by monitoring the amount of incoming SYN packets. Upon reaching the threshold, set at 20 SYN packets, a new cowrie honeypot container is deployed via docker. If after 3 seconds the SYN Count is at 0, an active cowrie honey pot container is removed. The reason for the 3 second buffer is to prevent it from scaling down aggressively - it makes sure that the SYN count is at 0 for the specified amount of time before removing a container. This ensures efficient resource management by making sure that the system doesn't consume more resources than is needed. By only deploying containers at the threshold and removing them during periods of idle, this minimizes the potential wasted resources. For HoneyCon, the maximum number of cowrie containers that can be active at one time is 50 to prevent host instability, once the limit is reached further deployment of new honeypots is ceased until existing containers are removed, this parameter can be changed depending on the requirements and system resources available.

3.2.3 Deception Layer

This final stage represents the response from the system, the previous stages work together to make this stage succeed. This is done by creating honeypot containers depending on the traffic received. As shown in Figure 3.2.3 below, the 'docker ps' command shows that the system successfully automated the deployment of multiple cowrie containers dependent on the data

received from the previous stages. The deployed containers are designed to act as sacrificial targets that will trap the attacker, shielding the legitimate network from the attack.

```
(osken-env) tomgx@ubuntu:~/typ$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
932e35f4e266	cowrie/cowrie:latest	"/cowrie/cowrie-env/..."	2 seconds ago	Up 1 second	2223/tcp, 0.0.0.0:2227->2222/tcp, :::2227->2222/tcp	cranky_jennings
9882ba6b9781	cowrie/cowrie:latest	"/cowrie/cowrie-env/..."	5 seconds ago	Up 3 seconds	2223/tcp, 0.0.0.0:2226->2222/tcp, :::2226->2222/tcp	pensive_hodgkin
1cea990ef591	cowrie/cowrie:latest	"/cowrie/cowrie-env/..."	7 seconds ago	Up 5 seconds	2223/tcp, 0.0.0.0:2225->2222/tcp, :::2225->2222/tcp	romantic_neumann
63e49ef05630	cowrie/cowrie:latest	"/cowrie/cowrie-env/..."	9 seconds ago	Up 7 seconds	2223/tcp, 0.0.0.0:2224->2222/tcp, :::2224->2222/tcp	goofy_lichterman
f2de8fdb1133	cowrie/cowrie:latest	"/cowrie/cowrie-env/..."	12 seconds ago	Up 10 seconds	2223/tcp, 0.0.0.0:2223->2222/tcp, :::2223->2222/tcp	compassionate_satoshi
99459c18b971	cowrie/cowrie:latest	"/cowrie/cowrie-env/..."	16 seconds ago	Up 14 seconds	0.0.0.0:2222->2222/tcp, :::2222->2222/tcp, 2223/tcp	inspiring_morse

Figure 3.2.3: This shows the deployment of multiple containers.

Resourceful Deception

With these decoy containers being deployed dynamically, it ensures that the defensive response corresponds with the current threat level and doesn't use up unnecessary resources. This makes the system more robust to high burst attacks, allowing the system to scale up and down as needed.

3.3 Implementation Challenges

This section will cover the challenges I went through with HoneyCon and the decisions made to overcome them.

3.3.1 Proper Network Redirection

HoneyCon successfully automates the dynamic scaling of docker container deployment through the detection of SYN floods through the use of Mininet. However, a significant technical hurdle remains in regards to redirecting malicious traffic to the cowrie containers that are deployed. In the current version of HoneyCon, while the orchestration layer [Section 3.2.2] deploys decoy cowrie docker containers based on the volume of incoming SYN packets, the system lacks the working mechanism to forward the malicious traffic to the deployed containers. A potential solution to achieve this would be to use a specialized honeypot like Glutton [12]. Cowrie focuses on SSH session interaction, whilst Glutton is a low-interaction honeypot that intercepts network traffic. With Glutton's ability to handle raw network streams, this would allow HoneyCon to the OSKen controller to redirect traffic into a Glutton instance, therefore the objective of redirecting malicious traffic to the appropriate honeypot would be achieved. [Section 1.3]

3.3.2 Framework Deprecation

A major obstacle that was encountered during the development of HoneyCon was the compatibility of the Ryu SDN controller [11] and the new versions of Python. With Python 3.12, the 'distutils' module was removed, which would cause critical errors during the installation of the Ryu package. This required the use of OSKen [13] which is a fork of Ryu but compatible with

new Python versions. This issue took a lot of troubleshooting and research but eventually allowed for HoneyCon to run. This challenge represents the importance of software supply-chain maintenance, especially in cyber security projects.

3.3.3 Justification for Hybrid Approach

The design choice of utilizing layer 4 network telemetry - specifically SYN packet frequency as the primary trigger for a layer 7 application response was to deviate from the traditional security systems approach of operating with a single layer. The idea was to employ a cross-layer orchestration strategy to mitigate the spin-up latency, to ensure that the decoy container is ready and operational by the time a malicious actor attempts to exploit the application layer - SSH authentication. However the practicality of this hybrid approach was less than ideal, during the actual implementation it became obvious that it would not work out as planned due to the incredible complexity of handing off an active TCP stream from the SDN switch to a containerized cowrie instance, requiring precise Network Address Translation (NAT). Attempting to achieve this would have introduced a plethora of problems, such as packet latency and instability that would potentially compromise the integrity of the deception layer. Ultimately, I decided to prioritize creating a robust proof of concept. The project successfully demonstrates the ability to scale up or down the amount of cowrie containers depending on the amount of traffic received. The current condition of the project provides a framework for future integration.

3.4 Resource Required

3.4.1 Hardware

Machine

- **CPU:** AMD Ryzen 7 9800X3D
- **RAM:** 64GB DDR5 6000MT/s
- **GPU:** AMD RX 7900XTX
- **STORAGE:** 1TB SSD

3.4.2 Software

Software Environment

- **Host OS:** Windows 11 24H2
- **Guest OS:** Ubuntu 24.04
- **Virtualization Software:** Virtual Box [14]
- **Python Version:** 3.12
- **SDN Controller:** OSKen (Fork of Ryu) [11]
- **Network Emulation:** Mininet
- **Containerization:** Docker
- **Honeypot Frameworks:** Cowrie
- **Testing Tools:** Hydra, Mininet

System Dependencies

- Open vSwitch
- Docker
- Docker-Compose

Python Packages

- os_ken
- mininet
- Docker

4 Development

This section will go into the development side of HoneyCon, going through the software that was used, to how everything works together.

4.1 System

For the system being used, a virtual machine was created on a host computer referenced in section 3.2.1. The host computer provided more than enough computing power for HoneyCon. The virtual machine was allocated with 22 GB of RAM, 6 processors and 42 GB of storage. Ubuntu 22.04 was used as the operating system; it is widely used and a stable platform for development, making it a suitable choice. The virtual machine was created on VirtualBox, this allowed for easily creating the virtual machine without complex configurations that other softwares impose.

4.1.1 Tools and Packages

The following were utilized for the development and deployment of HoneyCon:

- **Docker:** `sudo apt install docker`
- **Docker Compose:** `sudo apt install docker-compose`
- **Mini-net:** `sudo apt install mininet`
- **OS-Ken:** <https://github.com/openstack/os-ken>
- **Hydra:** `sudo apt install hydra`

Python Libraries:

- `pip install os-ken`
- `pip install eventlet`

Dependencies:

- `sudo apt install curl wget apt-transport-https lsb-release`
- `sudo apt install openvswitch-switch`

Essential Commands Used:

- `source /home/tomgx/osken-env/bin/activate`
- `osken-manager controller.py`
- `python honeypot-scaling.py`
- `sudo mn --custom topology.py --topo topology --controller remote --link tc`
- `h2 hping3 -S --flood -V -p 22 10.0.0.1`
- `docker stop $(docker ps -a -q)`
- `docker ps`
- `docker-compose up`
- `hydra -l root -P words.txt ssh://127.0.0.1:2222`

4.2 Honeypot Deployment with Docker

HoneyCon utilizes Docker to deploy containerized honeypots; docker provides a lightweight and efficient environment for virtualization. With each honeypot being containerized, HoneyCon ensures that each instance is run in isolation from the host. As docker is very lightweight, running multiple honeypot containers is not a problem, as can be seen in figure 4.2 below.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
2e3ce149d231	tender_edison	0.00%	41.48MiB / 21.78GiB	0.19%	4.47kB / 0B	0B / 24.6kB	1
ed8d1661688a	cranky_swanson	0.00%	41.47MiB / 21.78GiB	0.19%	4.47kB / 0B	0B / 24.6kB	1
7e2c39bb9da8	modest_liskov	0.00%	41.47MiB / 21.78GiB	0.19%	4.68kB / 0B	0B / 24.6kB	1
31f59bfla37d	agitated_pare	0.00%	41.47MiB / 21.78GiB	0.19%	4.88kB / 0B	0B / 24.6kB	1
e123ccfb7d07	serene_hamilton	0.00%	41.47MiB / 21.78GiB	0.19%	4.88kB / 0B	0B / 24.6kB	1
6b1d6af2c929	trusting_maxwell	0.00%	41.48MiB / 21.78GiB	0.19%	4.98kB / 0B	0B / 24.6kB	1
766dbcd3b660	adoring_northcutt	0.00%	41.47MiB / 21.78GiB	0.19%	4.98kB / 0B	0B / 24.6kB	1
77c832d508ce	typ_cowrie1_1	0.00%	68.05MiB / 21.78GiB	0.31%	216kB / 191kB	41.5MB / 270kB	1

Figure 4.2: A terminal output from running: docker stats

The primary honeypot used in HoneyCon is Cowrie, this honeypot is a medium to high-interaction SSH and Telnet honeypot. It is used specifically for capturing brute-force login attempts and command execution [7]. Cowrie is deployed by using the official docker image: cowrie/cowrie:latest, each container is then launched with specific port mappings. Each container operated within its own namespace, with its own resources limits to make sure that in case of a successful mass scaled attack, it is all isolated to the containerized environment. In figure 4.2.1 below, I have provided logs of an attempted login for a honeypot instance.

```
osken-env tomx@ubuntu:~/typ$ docker-compose up
Starting typ_cowrie1_1 ... done
Attaching to typ_cowrie1_1
typ_cowrie1_1 | /cowrie/cowrie-env/lib/python3.11/site-packages/twisted/conch/ssh/transport.py:105: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.
typ_cowrie1_1 | hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
typ_cowrie1_1 | b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
typ_cowrie1_1 | /cowrie/cowrie-env/lib/python3.11/site-packages/twisted/conch/ssh/transport.py:112: CryptographyDeprecationWarning: TripleDES has been moved to cryptography.
typ_cowrie1_1 | hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
typ_cowrie1_1 | b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] Reading configuration from ['/cowrie/cowrie-git/etc/cowrie.cfg.dist']
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] Python Version 3.11.2 (main, Sep 14 2024, 03:00:30) [GCC 12.2.0]
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] Twisted Version 24.10.0
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] Cowrie Version 2.6.1
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] Loaded output engine: jsonlog
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [twisted.scripts.twistd_unix.UnixAppLogger#info] twistd 24.10.0 (/cowrie/cowrie-env/bin/python3 3.11.2) starting up.
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [twisted.scripts.twistd_unix.UnixAppLogger#info] reactor class: twisted.internet.epollreactor.EPollReactor.
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] CowrieSSHFactory starting on 2222
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [cowrie.ssh.factory.CowrieSSHFactory#info] Starting factory <cowrie.ssh.factory.CowrieSSHFactory object at 0x73f8093ceb90>
typ_cowrie1_1 | 2025-04-07T09:54:58+0000 [-] Ready to accept SSH connections
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.factory.CowrieSSHFactory] No moduli, no diffie-hellman-group-exchange-sha1
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.factory.CowrieSSHFactory] No moduli, no diffie-hellman-group-exchange-sha256
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.factory.CowrieSSHFactory] New connection: 172.29.0.1:46792 (172.29.0.2:2222) [session: e6e7fdb50e6b]
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [HoneyPotSSHTransport,0,172.29.0.1] Remote SSH version: SSH-2.0-OpenSSH 9.6p1 Ubuntu-3ubuntu13.9
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [HoneyPotSSHTransport,0,172.29.0.1] SSH client hash fingerprint: aae6b9604f6f3356543709a376d7f657
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.transport.HoneyPotSSHTransport#debug] kex alg=b'curve25519-sha256' key alg=b'ssh-ed25519'
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.transport.HoneyPotSSHTransport#debug] outgoing: b'aes128-ctr' b'hmac-sha2-256' b'none'
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.transport.HoneyPotSSHTransport#debug] incoming: b'aes128-ctr' b'hmac-sha2-256' b'none'
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.transport.HoneyPotSSHTransport#debug] NEW KEYS
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-userauth'
typ_cowrie1_1 | 2025-04-07T09:55:17+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'none'
typ_cowrie1_1 | 2025-04-07T09:55:22+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' trying auth b'password'
typ_cowrie1_1 | 2025-04-07T09:55:22+0000 [HoneyPotSSHTransport,0,172.29.0.1] Could not read etc/userdb.txt, default database activated
typ_cowrie1_1 | 2025-04-07T09:55:22+0000 [HoneyPotSSHTransport,0,172.29.0.1] login attempt [b'root'/b'root'] failed
typ_cowrie1_1 | 2025-04-07T09:55:23+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' failed auth b'password'
typ_cowrie1_1 | 2025-04-07T09:55:23+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
```

Figure 4.2.1: Terminal output of docker-compose up. Shows unauthorized login at the end of log.

Hydra can be utilized for brute-forcing passwords, running hydra allows me to analyze the logs of the honeypots and see how the system handles the volume.

```

(osken-env) tomgx@ubuntu:~/typs$ hydra -l root -P words.txt ssh://127.0.0.1:2222
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-04-07 08:22:27
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 466550 login tries (l:1/p:466550), ~29160 tries per task
[DATA] attacking ssh://127.0.0.1:2222/
[2222][ssh] host: 127.0.0.1 login: root password: 1st
[2222][ssh] host: 127.0.0.1 login: root password: 2
[2222][ssh] host: 127.0.0.1 login: root password: 1080
[2222][ssh] host: 127.0.0.1 login: root password: 6c
[2222][ssh] host: 127.0.0.1 login: root password: 10-point
[2222][ssh] host: 127.0.0.1 login: root password: 10th
[2222][ssh] host: 127.0.0.1 login: root password: 12-point
[2222][ssh] host: 127.0.0.1 login: root password: 20
[2222][ssh] host: 127.0.0.1 login: root password: 2nd
[2222][ssh] host: 127.0.0.1 login: root password: 30-30
[2222][ssh] host: 127.0.0.1 login: root password: 11-point
[2222][ssh] host: 127.0.0.1 login: root password: 18-point
[2222][ssh] host: 127.0.0.1 login: root password: 20-point
[2222][ssh] host: 127.0.0.1 login: root password: 16-point
[2222][ssh] host: 127.0.0.1 login: root password: 2,4,5-t
[2222][ssh] host: 127.0.0.1 login: root password: 2,4-d

```

Figure 4.2.2: Running Hydra to display brute-force login attempt.

4.3 Traffic Management with OpenFlow

Within the simulated network, OpenFlow is used, OpenFlow controls network traffic through a controller, dictating the behaviour of switch through flow rules. HoneyCon makes use of the OSKen SDN Controller, managing traffic across the network. The network packets are captured using mechanisms such as `@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)`. In this scenario, the `EventOFPPacketIn` is programmed to detect specific patterns in traffic, TCP SYN packets that are targeting SSH ports. When these packets are received in a large volume in a short period of time, it is indicative of a potential brute-force attack. Once these malicious packets have been identified, the controller installs rules to redirect the attack from the original target to one of the many honeypots that have been deployed. The controller also monitors the amount of incoming SYN packets, this is crucial for the next section which will go into the automation and scaling of these honeypots.

```

if tcp_pkt and ip_pkt:
    # Only redirect if it's a SYN packet [0x02]
    if tcp_pkt.bits == 0x02:
        self.logger.info(f"MALICIOUS SYN: {ip_pkt.src} -> Triggering Honeypot Redirection")
        self.increment_syn_counter()

```

Figure 4.3: Showing the detection of incoming SYN packets, logging their source IP, and incrementing a counter for how many packets received.

4.4 Automation and Scaling

One of the main objectives of HoneyCon is to be able to dynamically scale the honeypots based on the observed network traffic conditions. Traditional honeypot frameworks deploy a static amount of honeypots that can end up consuming a large amount of resources. On the other hand, HoneyCon monitors incoming traffic in real-time, automatically adjusting the amount of honeypots deployed depending on the network conditions. The automation logic is placed in the `honeypot-scaling.py` script that interfaces with the controller and docker. The script periodically gathers metrics that are maintained by the controller, this includes information on the number of SYN packets that are received and when the predefined threshold has been reached. The script then launches new containers and updates the flow rules to redirect the traffic to a random honeypot within the scope to spread out the load. An example command: `'h2 hping3 -S --flood -V -p 22 10.0.0.1'`. This performs a SYN flood attack from h2, this is to simulate a high volume Denial of Service Attack. The controller detects this burst of SYN packets, and in response the scaling script launches honeypots in retaliation as can be seen in figure 4.4 below.

```
(osken-env) tomgx@ubuntu:~/typ$ python honeypot-scaling.py
SYN Count: 22 | Active Containers: 1
DEPLOYED: ef0963abf96f on port 2223. Total: 2
SYN Count: 15 | Active Containers: 2
SYN Count: 13 | Active Containers: 2
SYN Count: 14 | Active Containers: 2
SYN Count: 19 | Active Containers: 2
SYN Count: 7 | Active Containers: 2
SYN Count: 20 | Active Containers: 2
DEPLOYED: 60e2eb825696 on port 2224. Total: 3
SYN Count: 10 | Active Containers: 3
SYN Count: 8 | Active Containers: 3
SYN Count: 3 | Active Containers: 3
SYN Count: 8 | Active Containers: 3
SYN Count: 0 | Active Containers: 3
REMOVED: Honeypot on port 2224. Total: 2
SYN Count: 0 | Active Containers: 2
REMOVED: Honeypot on port 2223. Total: 1
SYN Count: 0 | Active Containers: 1
REMOVED: Honeypot on port 2222. Total: 0
```

Figure 4.4: Honeypot containers being deployed or removed in relation to scaling.

Each honeypot is capable of capturing detailed logs on what the user is attempting to do in the isolated environment without them ever knowing about it. The honeypots are then removed once the threat is no longer present, avoiding unnecessary resource consumption.

5 Testing and Evaluation

This section will go into detail on the testing procedures that have been used to determine whether HoneyCon has been successful in terms of functionality and performance. Testing and evaluation of HoneyCon is crucial to ensure that the system has met the objectives of being a scalable system capable of managing attacks efficiently while being reliable. While it wasn't possible to complete the TPC hand-off as mentioned in my implementation, the scaling of the honeypots in response to the traffic conditions was successful.

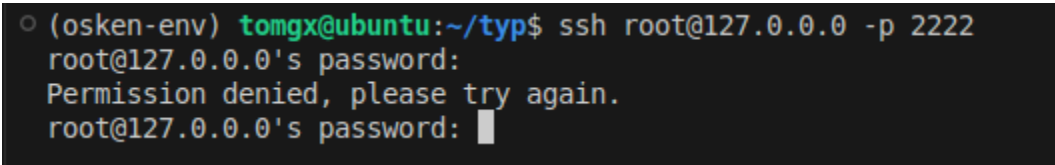
5.1 Functional Testing

In terms of functional testing, this is to ensure that everything works as expected, all individual components are to be tested.

- **Functionality of Cowrie:**

Manual connection to the honeypot created. Using `ssh root@127.0.0.0 -p 2222`

The expected outcome of a successful connection to the honeypot would be a fake login prompt. If a login attempt is made, permission is denied and a log in made.

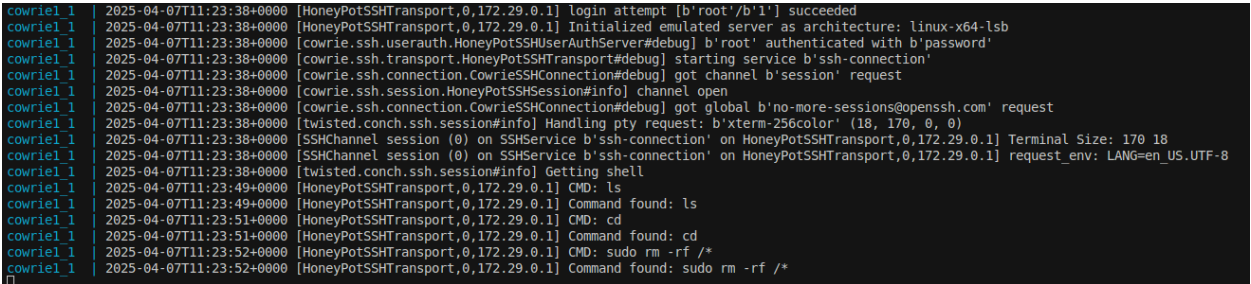


```
(osken-env) tomgx@ubuntu:~/typ$ ssh root@127.0.0.0 -p 2222
root@127.0.0.0's password:
Permission denied, please try again.
root@127.0.0.0's password: █
```

Figure 5.1: Successful connection to honeypot with manual login attempt.

- **Simulated attack:**

Upon a successful login attempt where the attacker attempts to execute any commands, all of that is logged and execution of the commands are isolated to the container.



```
cowrie 1 2025-04-07T11:23:38+0000 [HoneyPotSSHTransport,0,172.29.0.1] login attempt [b'root'/b'l'] succeeded
cowrie 1 2025-04-07T11:23:38+0000 [HoneyPotSSHTransport,0,172.29.0.1] Initialized emulated server as architecture: linux-x64-lsb
cowrie 1 2025-04-07T11:23:38+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] b'root' authenticated with b'password'
cowrie 1 2025-04-07T11:23:38+0000 [cowrie.ssh.transport.HoneyPotSSHTransport#debug] starting service b'ssh-connection'
cowrie 1 2025-04-07T11:23:38+0000 [cowrie.ssh.connection.CowrieSSHConnection#debug] got channel b'session' request
cowrie 1 2025-04-07T11:23:38+0000 [cowrie.ssh.session.HoneyPotSSHSession#info] channel open
cowrie 1 2025-04-07T11:23:38+0000 [cowrie.ssh.connection.CowrieSSHConnection#debug] got global b'no-more-sessions@openssh.com' request
cowrie 1 2025-04-07T11:23:38+0000 [twisted.conch.ssh.session#info] Handling pty request: b'xterm-256color' (18, 170, 0, 0)
cowrie 1 2025-04-07T11:23:38+0000 [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,0,172.29.0.1] Terminal Size: 170 18
cowrie 1 2025-04-07T11:23:38+0000 [SSHChannel session (0) on SSHService b'ssh-connection' on HoneyPotSSHTransport,0,172.29.0.1] request_env: LANG=en_US.UTF-8
cowrie 1 2025-04-07T11:23:38+0000 [twisted.conch.ssh.session#info] Getting shell
cowrie 1 2025-04-07T11:23:38+0000 [HoneyPotSSHTransport,0,172.29.0.1] CMD: ls
cowrie 1 2025-04-07T11:23:49+0000 [HoneyPotSSHTransport,0,172.29.0.1] Command found: ls
cowrie 1 2025-04-07T11:23:51+0000 [HoneyPotSSHTransport,0,172.29.0.1] CMD: cd
cowrie 1 2025-04-07T11:23:51+0000 [HoneyPotSSHTransport,0,172.29.0.1] Command found: cd
cowrie 1 2025-04-07T11:23:52+0000 [HoneyPotSSHTransport,0,172.29.0.1] CMD: sudo rm -rf /*
cowrie 1 2025-04-07T11:23:52+0000 [HoneyPotSSHTransport,0,172.29.0.1] Command found: sudo rm -rf /*
```

Figure 5.1.1: Attempted command execution:

Command found: ls

Command found: cd

Command found: sudo rm -rf /*

- **OpenFlow routing:**

The pingall command was executed on mininet to ensure that the hosts could communicate with each other. The expected outcome of this test is to ensure that 0% of packets are dropped. It is important to make sure that there are no connectivity issues.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
```

Figure 5.1.2: pingall command in mininet to show ping reachability.

- **HoneyPot scaling:**

Automated scaling was tested by using Mininet, this provided the ability to simulate SYN flood attacks. Testing the system's response to these attacks was crucial to see whether the system was scaling correctly in response to the traffic. It is important for the honeypots to know whether to scale up or down, to make sure that there are no idling honeypots that are taking up resources wastefully. This can be seen in [Figure 4.4].

To conclude this section, the Cowrie honeypot functions as expected, any interaction with the honeypot is then logged and can be viewed with the command 'docker-compose up', the hosts can communicate with each other, and the honeypot scaling functions dynamically. When the SYN count drops to 0 for 3 seconds, the system scales down by removing honeypot containers.

5.2 Performance Evaluation

Evaluating the performance of HoneyCon will provide important data on how the system performs under different stress loads. Using 'docker stats' outputs the relevant resource utilization information. Running attack simulations while this is running will output the results of the system's efficiency. Figure 5.2 below displays 10 honeypot containers running after running simulated attacks. The results show that the system is efficient in scaling honeypots.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
d2abea134af0	friendly_shockley	0.00%	41.47MiB / 21.78GiB	0.19%	3.24kB / 0B	0B / 24.6kB	1
5cb0b2d942e0	objective_shannon	0.00%	41.6MiB / 21.78GiB	0.19%	2.98kB / 0B	0B / 24.6kB	1
917eb70c9181	happy_wright	0.00%	41.47MiB / 21.78GiB	0.19%	2.98kB / 0B	0B / 24.6kB	1
cc0f0bd021c1	magical_easley	0.00%	41.48MiB / 21.78GiB	0.19%	2.98kB / 0B	0B / 24.6kB	1
affd770e4645	great_tharp	0.00%	41.48MiB / 21.78GiB	0.19%	2.87kB / 0B	0B / 24.6kB	1
d303ab96b287	strange_ptolemy	0.00%	41.47MiB / 21.78GiB	0.19%	2.8kB / 0B	0B / 24.6kB	1
53df8e298801	xenodochial_ishizaka	0.00%	41.47MiB / 21.78GiB	0.19%	2.8kB / 0B	0B / 24.6kB	1
a33d2e8a73b3	angry_kalam	0.00%	41.46MiB / 21.78GiB	0.19%	2.7kB / 0B	0B / 24.6kB	1
b34fd1f6e8e6	amazing_wescoff	0.00%	41.47MiB / 21.78GiB	0.19%	2.63kB / 0B	0B / 24.6kB	1
7f1087897a66	stoic_hawking	0.00%	41.48MiB / 21.78GiB	0.19%	2.52kB / 0B	0B / 24.6kB	1

Figure 5.2: Shows the resources being used per honeypot.

Referring back to Figure [4.4], once the attack has halted, the honeypots will be scaled down 1 by 1 until it has reached 0, the system will then idle waiting for any attacks to scale up honeypots again.

To ensure smoothing scaling of honeypots, the system checks periodically for attacks, honeypots are then created or removed without any delays or spikes in performance.

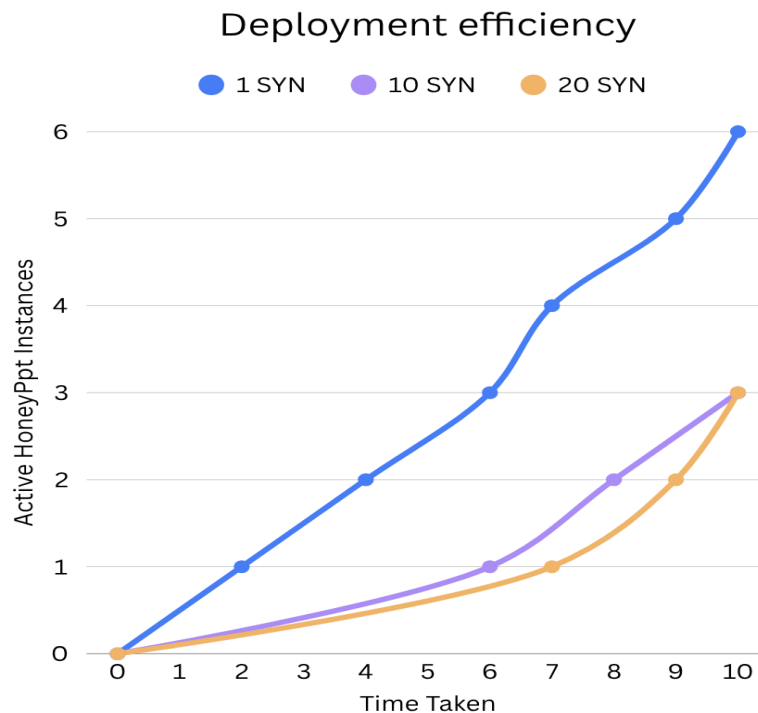


Figure 5.2.1: Graph to show the efficiency of HoneyPot deployment

In Figure 5.2.1, the graph demonstrates the orchestration latency, the lower the SYN count threshold is, the faster the honeypot instances are deployed. The results are taken over a 10 second time frame. The results may vary on a different system; the test I carried out was on a Virtual Machine that was suffering from some bottlenecks.

6 Conclusion

6.1 Review of Project Aims

The aim of this project was to implement a system that can scale honeypots dynamically, responding to the network conditions by deploying containerized honeypots. I believe that the HoneyCon project has been successful in creating a system that can detect SSH brute-force attacks, SYN flood attacks, and automates the deployment of Cowrie honeypots in response with the integration of Docker, OpenFlow, and OSKen (a ryu fork). The project effectively identifies layer 4 SYN flood attacks, then automates the deployment and or the removal of cowrie containerized honeypots. HoneyCon demonstrates that resource overhead commonly associated with the traditional static honeypots systems can be significantly reduced with the use of a dynamically scaling system that responds to varying traffic conditions.

6.2 Future Work

While HoneyCon provides a proof of concept for a robust system, there are a number of improvements that can be made to HoneyCon, one of them by using a more advanced system, integrating machine learning that would prevent a lot of false flags. As of right now HoneyCon works on a simple threshold system to detect SYN packets which is not optimal in the long run. Another important improvement that could be made is by implementing additional types of honeypots beyond Cowrie, as of right now my system is only limited to one type, making it very limited in terms of functionality. Increasing the range of honeypots would allow for better monitoring of a wider range of attacks. Another potential improvement that could be added is a web-based dashboard for monitoring the system, this would provide real-time data and the ability to manage every honeypot deployed, making this a quality of life improvement for when a larger-scale deployment is in progress. A critical step for future work would be refining the NAT and TCP sequence synchronization, this was identified as a technical hurdle during the implementation process due to a high level of complexity, the idea of handing off active data streams from the SDN switch to the cowrie honeypots that have been deployed.

6.3 Lessons Learned

The project involved a lot of background knowledge into how each system worked, one of the most important lessons learned is making sure that each component of the system interacted with each other correctly. This included making sure that Mininet, OSKen, and Docker could effectively communicate and exchange the correct information between each other. After working with Docker throughout the project, I have found myself more proficient in how it

works. I believe that this is true for all of the systems that I worked with to make HoneyCon function. Navigating the deprecation of legacy software and adapting to the constraints of Python 3.12, such as using OSKen - a Ryu fork that is maintained as a separate branch.

6.4 Changes from the Proposal

The project was initially planned to have multiple honeypot types, the reason for only using Cowrie for the project is due to some roadblocks that I encountered during the process of completing the project. To ensure that a stable and robust system was developed, the scope of the project narrowed towards providing a framework for scaling cowrie instances dependent on the amount of SYN packets received, rather than an unstable variety of honeypots.

6.5 Final Remarks

Having completed this dissertation, the journey of developing the honeypot infrastructure has been a technical challenge. I have gained a lot of new knowledge from working on the project, setting up tools like Docker, OpenFlow, OSKen on a virtual machine, making sure everything works together smoothly was a great challenge to begin with. I remember struggling to get originally Ryu to work, due to 'pip install distutils' no longer existing for Python version 3.12, I was stuck for a long time trying to figure out why I couldn't get Ryu to install. Thankfully a more up to date, and maintained version of Ryu (OSKen) exists, this allowed me to proceed with the project. I have learnt through lots of trial and error, the most complex issues can be solved - as long as there is enough time. I also truly believe that this project could have been magnitudes better than what it currently is if it weren't for roadblocks that I encountered during the period of this project. Technical constraints limited the final scope of redirecting the malicious packets to the containerized honeypots, but the resulting framework that has been created through all of this provides a scalable foundation that can be improved on.

Appendix A

Reproducibility repository

[Github link](#)

Appendix B

Project Proposal

Final Year Project Proposal - HoneyCon

Abstract

The proposed project is to develop a scalable, dynamic honeypot infrastructure through the use of Docker containers and OpenFlow. Honeypots are vital tools for cybersecurity, simulating real systems to attract and analyze attacks. Traditional honeypot systems are limited by their static nature (fixed configurations - making them predictable) and they also lack scalability (resource constraints - deploying multiple traditional honeypots can require a significant amount of resources). HoneyCon seeks to overcome these limitations by creating a system that can dynamically scale honeypot instances in response to real-time malicious activity. OpenFlow will allow traffic to be routed to honeypots. By the end of the project, HoneyCon will provide a scalable solution for capturing a range of attacks efficiently while reducing resource waste.

Introduction

Honeypots are network decoys that are designed to attract attackers and prevent them from getting to important information, they allow the observation of attack behaviors in a controlled environment. As networks become larger and attacks more complex, traditional honeypots face scalability issues, it then becomes difficult to handle higher traffic volumes or distributed attacks that can cause serious issues. [5]

This project aims to address these limitations by combining containerization and SDN technologies. Docker containers will allow dynamic scaling of honeypot instances. OpenFlow will be used for traffic routing, making sure that malicious traffic is redirected to the correct honeypot appropriate to the attack. The system will be capable of automatically adjusting to traffic loads, deploying new honeypots without manual intervention. [4]

By using open-source tools, this project aims to create a solution for scaling honeypot networks, improving the ability to capture and analyze a range of cyberattacks without making it too abstract.

Related Work

There are many honeypot solutions, looking at T-POT, it is a multi-honeypot platform - integrating multiple honeypot technologies. T-POT supports over 20 honeypots with various visualization options. It is also built on top of Docker containers which allows for easy deployment and scalability. One of the core features of T-POT is the ability to simulate different

services and protocols - SSH, HTTP, FTP. This allows for the capture of a wide range of attacks. T-POT is integral for the progression of cybersecurity; it offers valuable data of the mindset of attackers and the patterns they follow. [3]

The Proposed Project

Aims and Objectives

The primary aim of this project is to develop a dynamic, scalable honeypot infrastructure using Docker containers and OpenFlow. The key objectives are:

- **Deploy a honeypot network:** Using Docker to manage lightweight honeypots that can be scaled automatically in response to traffic.
- **Integrate OpenFlow for traffic management:** Use OpenFlow to dynamically redirect malicious traffic to the suitable honeypots.
- **Automate honeypot scaling:** Implement a system that monitors network traffic and scales honeypots up or down based on specific thresholds.
- **Capture and analyze attack data:** Enable logging of malicious activity to provide insights into attack patterns and methods.

Methodology

The project will be carried out in stages, starting with the design and development of a basic honeypot infrastructure using Docker. OpenFlow will be used to manage traffic routing, redirecting suspicious packets to honeypots.

1. **Docker:**
 - Use open-source honeypot systems (e.g., Dionaea, Cowrie) to create lightweight honeypot instances. [1]
 - Develop mechanisms to scale containers dynamically using Docker Swarm.
2. **OpenFlow Traffic Management:**
 - Configure OpenFlow rules to inspect incoming traffic and forward malicious packets to honeypots.
 - The OpenFlow controller will dynamically update flow tables based on traffic patterns, enabling real-time scalability. [2]
3. **Automation and Monitoring:**
 - Develop automation scripts to monitor network traffic and adjust the number of honeypot containers based on traffic volume.

4. Testing and Evaluation:

- Simulate various attack scenarios to evaluate the system's scalability and performance.
- Measure resource consumption and system response times under different traffic loads.

Programme of Work

The project will be divided into several phases over the course of the academic year:

- **Week 1 - Week 5**
 - Initial research and setup of Docker and OpenFlow environments.
 - Design and implementation of honeypot systems.
- **Week 5 - Week 10**
 - Implement traffic redirection using OpenFlow.
 - Develop the automation for scaling honeypot containers.
- **Week 10 - Week 15**
 - Testing of the system under simulated attack scenarios.
 - Collection and analysis of system performance data.
- **Week 15 - Week 20**
 - Final evaluation of the system, potentially performance benchmarking?.
 - Write-up of results and final dissertation submission.

Extension

Potentially testing against malware to see how it reacts to it. Adding more functionality to the HoneyCon project.

Resources Required

- **Hardware:** Access to a development machine capable of running Docker and SDN simulations. (My computer running Linux).
- **Software:** Open-source honeypot software (e.g., Dionaea), Docker, OpenFlow (e.g. Ryu), logging tools. (Docker Logs?)
- **Network Simulation Tools:** Tools such as Mininet to simulate network environments for testing honeypot systems.

References

- [1] Dionaea Honeypot: <https://github.com/DinoTools/dionaea> - Accessed 20th Oct 2024
- [2] OpenFlow Protocol Overview: <https://www.opennetworking.org/sdn-resources/openflow> - Accessed 20th Oct 2024
- [3] T-Pot CE (Honeypot Framework): <https://github.com/telekom-security/tpotce> - Accessed 20th Oct 2024
- [4] UCSD Honeypot Research: <https://cseweb.ucsd.edu/~savage/papers/Sosp05.pdf> - Accessed 20th Oct 2024
- [5] Honeypot(computing) [https://en.wikipedia.org/wiki/Honeypot_\(computing\)](https://en.wikipedia.org/wiki/Honeypot_(computing)) - Accessed 20th Oct 2024

References

- [1] Cisco (2024) Cisco Secure Firewall: First Line of Defense. Available at: <https://www.cisco.com/site/uk/en/learn/topics/security/what-is-a-firewall.html>
[Accessed: 3 April 2025].
- [2] GuardRails (2022) False Positives and False Negatives in Information Security. Available at: <https://www.guardrails.io/blog/false-positives-and-false-negatives-in-information-security/>
[Accessed: 3 April 2025].
- [3] Jacky (2023) What Are Honeydumps? Types, Benefits, Risks, and Best Practices. Available at: <https://www.sapphire.net/blogs-press-releases/what-are-honeydumps>
[Accessed: 3 April 2025].
- [4] Provos, N. (n.d.) Honeyd Frequently Asked Questions. Available at: <https://www.honeyd.org/faq/#what-is-honeyd>
[Accessed: 3 April 2025].
- [5] telekom-security (n.d.) GitHub - telekom-security/tpotce: 🍵 T-Pot - The All In One Multi Honeyd Platform 🐝. Available at: <https://github.com/telekom-security/tpotce>
[Accessed: 3 April 2025].
- [6] RedHat (n.d.) What is container orchestration? Available at: <https://www.redhat.com/en/topics/containers/what-is-container-orchestration>
[Accessed: 3 April 2025].
- [7] Oosterhof, M. (n.d.) Cowrie — cowrie 2.6.1 documentation. Available at: <https://docs.cowrie.org/en/latest/README.html#what-is-cowrie>
[Accessed: 3 April 2025].
- [8] Narmox (n.d.) Mininet Topology Visualizer. Available at: <http://mininet.spear.narmox.com>
[Accessed: 1 April 2025].
- [9] Docker (n.d.) *Docker Documentation*. Available at: <https://docs.docker.com>
[Accessed: 5 April 2025].
- [10] Mininet Project (n.d.) *Mininet: An Instant Virtual Network on your Laptop*. Available at: <http://mininet.org>

[Accessed: 28 March 2025].

[11] Ryu SDN Framework (n.d.) *Ryu SDN Framework Community Site*. Available at:
<https://ryu-sdn.org>

[Accessed: 25 March 2025].

[12] Glutton (n.d.) GitHub - Glutton : Generic Low Interaction Honeypot-. Available at:
<https://github.com/mushorg/glutton>

[Accessed: 12 April 2025].

[13] OSKen (n.d.) GitHub - os-ken: A component-based software defined networking framework in OpenStack. Available at: <https://github.com/openstack/os-ken>

[Accessed: 20 March 2025].

[14] VirtualBox (n.d.) virtualbox -Powerful open source virtualization. Available at:
<https://www.virtualbox.org/>

[Accessed: 20 March 2025].